Machine Learning Operations Applied to Development and Model Provisioning

Óscar A. Méndez[1](✉), Jorge Camargo[1], and Hector Florez[2]

[1] Universidad Nacional de Colombia, Bogota, Colombia
{omendeza,jecamargom}@unal.edu.co
[2] Universidad Distrital Francisco Jose de Caldas, Bogota, Colombia
haflorezf@udistrital.edu.co

Abstract. In the current era of software engineering, where Machine Learning (ML) plays a crucial role in technological innovation, effective implementation of development and operations practices is essential. The DevSecOps (Development, Security, Operations) approach has gained popularity due to its ability to integrate security and quality into every stage of the software development lifecycle. However, in the specific context of Machine Learning, the need arises for a specialized approach that takes into account the peculiarities of the models and algorithms used. Machine Learning Operations (MLOps), despite their relative novelty, seek to establish a framework to characterize the ML development life cycle, decouple it from software development, and ensure quality attributes such as scalability, maintainability, and security. This paper focuses on the exploration and application of Machine Learning Operations (MLOps) in the specific context of developing and provisioning machine learning models.

Keywords: Machine Learning · MLOps · DevSecOps · Data management · Technology innovation

1 Introducción

In modern industry, Machine Learning Operations (MLOps) has emerged as a key discipline, transforming how organizations develop, deploy, and maintain Machine Learning (ML) models. MLOps is a natural evolution of DevOps applied to the field of ML and addresses the need to integrate software engineering practices with the ML lifecycle. The importance of MLOps lies in its ability to standardize and optimize ML processes, allowing faster, more efficient, and scalable development of data-driven solutions [3]. The impact of MLOps on the industry is considerable, as it provides a framework to overcome common challenges in ML projects, such as reproducibility, technical debt management, and integration between data and operations teams. The novelty of MLOps lies in its holistic approach, which ranges from data management and experimentation to the deployment and monitoring of models in production [13]. Key

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
H. Florez and H. Astudillo (Eds.): ICAI 2024, CCIS 2236, pp. 73–88, 2025.
https://doi.org/10.1007/978-3-031-75144-8_6

concepts in MLOps include ML pipeline automation, continuous integration and delivery (CI/CD) applied to ML models, and monitoring and managing model performance in production environments. These practices not only improve the efficiency and quality of ML projects but also foster more effective collaboration between data science and operations teams [25].

This article highlights the need for a specialized approach in ML, underlining the particularities of the models and algorithms used in this field. Based on a detailed analysis, we argue how MLOps provides a robust framework for ML development, differentiating it from the conventional software development life cycle. These differences are essential to address specific challenges such as data management, security, and quality in ML.

The main goal is to examine how implementing MLOps can significantly improve ML development lifecycle management. It focuses on contributing to knowledge in the emerging field of MLOps and promoting the adoption of best practices in ML-based solutions. A methodology focused on the practical application of MLOps and its impact on the ML development lifecycle is used, highlighting the importance of an integrated approach that combines aspects of development, security, and operations (DevSecOps) in the context of ML. A differential element of this work is the case study implemented in a model already developed. This case study illustrates the practical application of MLOps in a real-world context and demonstrates how its implementation improves scalability, maintainability, and security in ML model development.

Therefore, technical debt is explored in the context of MLOps as a fundamental issue in the execution of the case study based on some previous documented experiences [22,24]. Technical debt, which is a concept taken from software development and adapted to ML, refers to suboptimal design or implementation decisions that, while they may speed up development in the short term, can cause significant and costly problems in the long term. In MLOps, technical debt can arise from various sources, such as data selection, model choice, or model lifecycle management. The research discusses how effective implementation of MLOps can help mitigate technical debt, thereby ensuring greater sustainability and efficiency in ML projects [22].

Regarding security and ethical issues [23], these aspects are of great importance in the development of ML. This work delves into how MLOps can improve the security of ML models, addressing topics such as data protection, privacy, and model integrity. Additionally, we discuss ethical challenges inherent to ML, such as bias in data and algorithms, and explore how MLOps practices can help identify and mitigate these issues, thereby promoting more responsible and ethical ML development [16,18].

Finally, automation and architecture in MLOps are critical to the scalability and maintainability of ML systems. Automation in MLOps, which covers from continuous integration and delivery to model monitoring and management, is essential to streamline the ML development lifecycle and to quickly respond to market changes and needs [8]. Furthermore, a well-designed architecture is crucial for the scalability and maintainability of ML systems. This paper examines

how an effective MLOps architecture not only supports the expansion and adaptability of ML systems, but also ensures that these systems are robust, secure, and easy to maintain over time [14].

2 MLOps

In the conceptualization stage of ML projects [23], which is the first point in the ML life cycle [27], MLOps plays a crucial role. Here, the implementation of MLOps ensures the creation of clear and scalable processes from the beginning of the project. This includes precisely defining project objectives and scope, selecting appropriate data, and defining clear and achievable success metrics. By establishing a solid foundation in this initial phase, MLOps facilitates smoother and more efficient development across the stages of the ML lifecycle [19]. MLOps follows the following phases:

- **Project Conceptualization.** In this initial phase, MLOps defines the goals, scope, and requirements of the ML project. Implementing MLOps ensures clear and scalable processes from the beginning, including selecting appropriate data and defining success metrics [23].
- **Model Development.** During model development, MLOps facilitates integration and collaboration between data scientists and operations teams. This includes implementing continuous integration practices, automated testing, and utilizing versioning tools to ensure the quality and reproducibility of code and models [19].
- **Model Validation.** MLOps provides frameworks for automated testing and evaluation of model quality. It focuses on the accuracy, performance, and generalization of the model to ensure that it meets the stated objectives [19].
- **Deployment and Operation.** The deployment phase benefits significantly from MLOps, especially in implementing continuous delivery practices. MLOps enables efficient and secure deployment of models into production environments, managing the necessary infrastructure and ensuring optimal compatibility and performance [19].
- **Monitoring and Maintenance.** MLOps is essential in continuously monitoring deployed models, identifying and addressing performance issues, drifts, and other data changes. The practice of MLOps ensures that models are kept up to date and continually improved over time [5,7,19].
- **Security.** Implementing MLOps at each stage of the ML lifecycle plays a critical role in managing the security of models [8]. From the initial conceptualization phase, MLOps focuses on establishing a solid security foundation, focusing on the secure selection of data and the definition of metrics that include security aspects [18,24]. During model development, MLOps integration ensures that security practices are intrinsically integrated into the development process, through automated testing and versioning tools, thus strengthening the security of the code and models [16]. In the validation stage, MLOps focuses on ensuring that models are not only accurate and efficient but also secure and resilient to potential vulnerabilities. The deployment and

operation phase, under the MLOps approach, highlights the importance of deploying models in production environments safely, and properly managing the infrastructure to prevent security risks. Lastly, in the ongoing monitoring and maintenance of models, MLOps facilitates the proactive identification of security issues, ensuring that models remain up-to-date and protected against emerging threats. Taken together, MLOps not only improves the efficiency and effectiveness of ML models but also establishes a comprehensive and consistent security culture at every stage of the ML lifecycle [12,28].

3 Proposed Approach

This approach illustrates the application and valorization of MLOps processes in the deployment of machine learning models, taking as an example a specific model based on Caenorhabditis elegans (C. elegans) [9,10]. The analysis is immersed in the previously presented design, to demonstrate how MLOps optimizes and improves the different stages of the life cycle of an ML project, from its conceptualization to its maintenance and operation. Although the case of the C. elegans model is specific, the main focus is on the ability of MLOps to effectively handle the complexity inherent in these projects, highlighting its contribution to improving efficiency and reproducibility.

Furthermore, the case study emphasizes the critical importance of security within the MLOps framework. Treated as an essential and integrated component of the MLOps workflow, security plays a critical role in protecting data and maintaining model integrity throughout all its phases. This approach not only improves the robustness of the MLOps process but also demonstrates its overall effectiveness, underlining its usefulness in the effective management of ML projects in complex and varied contexts, as evidenced by the application of the C. elegans models.

The approach addresses the lack of a formal MLOps process in managing an ML model based on C. elegans. This lack has generated significant challenges in critical aspects of the model lifecycle, impacting key areas such as model and data versioning, security, scaling, and monitoring, as well as continuous integration. Without a well-structured MLOps framework, problems have been faced in maintaining model consistency and traceability throughout its iterations, leading to inefficiencies and potential errors.

Security stands out as an aspect particularly affected by the lack of a standardized approach to MLOps, which can result in significant vulnerabilities in the protection of sensitive data. Additionally, the lack of consolidated practices in MLOps has limited the ability to efficiently scale the model, restricting its applicability and performance in broader contexts. Continuous integration and delivery, essential for the agility and timely updating of the model, have also been weakened, preventing an adequate response to changes and emerging needs. This set of problems affects not only the effectiveness of the current model but also raises significant concerns for its long-term sustainability and evolution.

The main objective is to establish and implement a formal MLOps process for the efficient and secure management of the machine learning model taking

as a reference case the C. elegans model, developed by Hernandez et al., [9]. This objective seeks to address and overcome identified challenges in the model lifecycle, including improving data and model versioning, security, scaling, monitoring, and continuous integration. Through the application of standardized and consolidated MLOps practices, the study aims to increase the efficiency, scalability, and reproducibility of the model, ensuring its applicability and performance in broader contexts. In addition, it focuses on strengthening the security of the model and sensitive data, and improving the responsiveness to changes and emerging needs, thus contributing to the sustainability and long-term development of the project.

3.1 Selected Technologies

In the development of the MLOps approach, various specialized tools were selected and used, each designed to optimize and facilitate different stages of the life cycle of ML models. This integrated approach ensures that from conception to implementation, every step of the process is handled with best practices and appropriate technologies to ensure the efficiency, scalability, and security of the models. The choice of each tool was not random but was based on its ability to address specific challenges within the MLOps framework, thus providing a robust and well-articulated solution that improves collaboration between teams, automates critical processes, and maintains the quality and accuracy of the model throughout its life cycle [20,26].

Furthermore, strategic use of these tools not only makes it easier to manage the model development and operation process but also helps to overcome common obstacles such as large-scale data management, complex model experimentation and tuning and integration, and continuous updating of models in production environments [26]. This meticulously curated technology infrastructure allows the team to focus on innovation and continuous improvement while minimizing risks and maximizing productivity. Thus, the approach demonstrates how an appropriate selection of MLOps tools and technologies can transform the development and implementation of machine learning-based solutions in an enterprise environment [1]. The selected technologies are:

- **GitLab**: Used as the central platform for versioning code and models, as well as to facilitate continuous integration and continuous delivery (CI/CD). GitLab enables efficient management of development and deployment workflows, integrating automation features that accelerate production and ensure software quality.
- **MLflow**: Used for life cycle management of ML models. MLflow provides an interface for logging experiments, managing runs, and packaging and serving models, making it simple to track model performance and facilitating the tuning and improvement process.
- **DVC (Data Version Control)**: Implemented for version management of large data sets, which helps to maintain the traceability and reproducibility of experiments by facilitating version control in data science environments, similar to how Git handles source code.

- **SonarQube and Snyk**: These tools are used to ensure the quality of the code and its security. SonarQube analyzes and detects quality issues that can affect the functionality and maintainability of the code, while Snyk focuses on identifying and mitigating vulnerabilities related to security dependencies.
- **Pre-commit, Black, Flake8, Pytest**: This set of development tools is used to ensure high-quality standards in the code. Pre-commit automates code verification before each commit, Black reformats code for consistency, Flake8 analyzes code for errors, and Pytest makes unit testing easy.
- **Python**: Selected as the primary programming language due to its wide adoption in the data science community and its robust support for machine learning and data processing techniques.
- **DagsHub**: Platform used to host and manage data repositories and ML models, facilitating collaboration and versioning. DagsHub integrates with DVC and MLflow, providing a complete solution for managing data science and machine learning projects.

Each tool has been carefully integrated to form a coherent ecosystem that supports and enhances MLOps, from the development phase to production and continuous monitoring, thus ensuring the delivery of ML-based solutions that are not only effective but also safe and sustainable

3.2 Reference Architecture

In this project, the Google reference architecture [8] was adopted, which is widely recognized for its ability to efficiently manage the life cycle of ML models in an environment of production. This architecture is designed to promote robust continuous integration and delivery (CI/CD) practices specifically tailored for ML. Google's reference architecture addresses critical aspects such as model training, validation, deployment, and continuous performance monitoring, allowing for iterative updates and improvements without disruption to active services. By integrating these operations into a continuous flow, the architecture facilitates a smooth transition of models from development to production, optimizing collaboration between data and operations teams [2,6,21].

Using this reference architecture not only provides a framework for process automation but also sets standards for rigorous monitoring and validation of models in production environments. By following this framework, the team can ensure that models are scalable, reproducible, and robust to changes in data or the operating environment. Additionally, Google's architecture helps to quickly identify and address any degradation or failures in the model, thereby ensuring reliability and operational efficiency. This systematic approach not only improves the quality and accuracy of model predictions but also optimizes resources and reduces the time needed to bring innovations from conception to market.

3.3 Suggested Pipeline

Within the MLOps, a pipeline was implemented composed of several fundamental phases for the development, validation, and implementation of machine

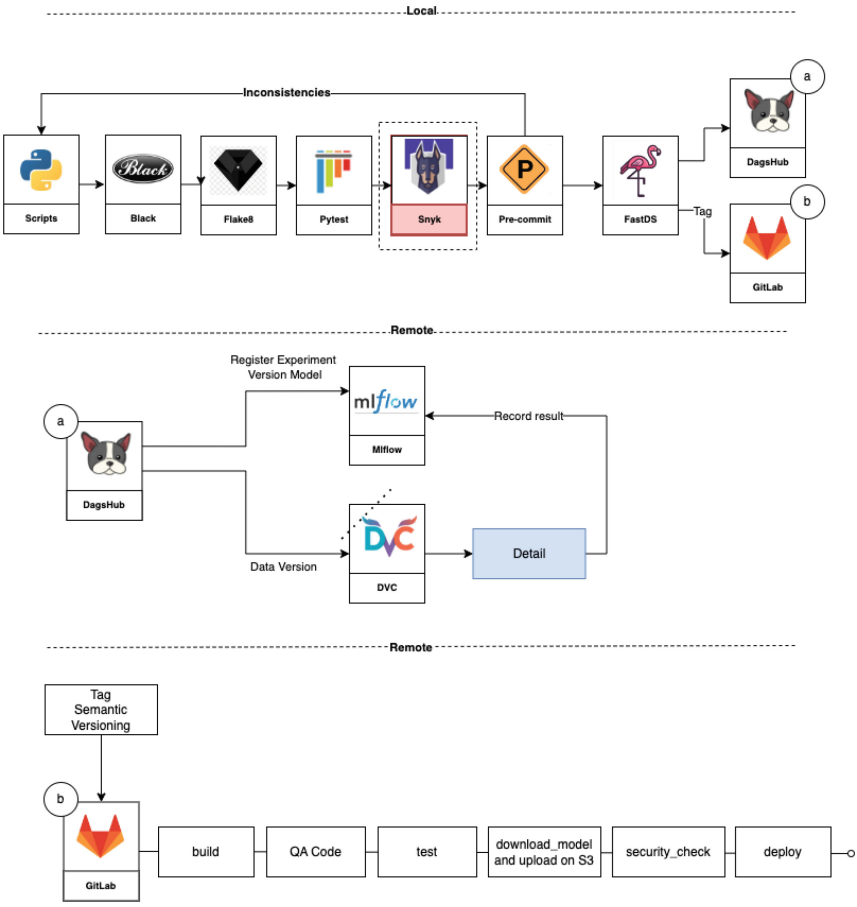

Fig. 1. General pipeline.

learning models. Figure 1 presents the pipeline. Initially, in the local environment, Python scripts were used to code formatting using Black, thus ensuring a coherent and standardized code base. Code quality was evaluated with tools such as Flake8 and unit tests were managed through Pytest, ensuring the robustness of the developed code. Additionally, security review mechanisms were integrated with Snyk, and code reviews before commits were automated using Pre-commit. This process culminated with the integration and versioning of the changes in the GitLab platform, which also served as a bridge to the remote environment for the registration and versioning of models through MLflow and data management through DVC, reinforcing collaboration and the traceability of the project.

In the transition to the remote environment (see Fig. 2), the pipeline took advantage of MLflow's capacity for exhaustive monitoring of experiments and model versioning, allowing meticulous analysis of the results and model itera-

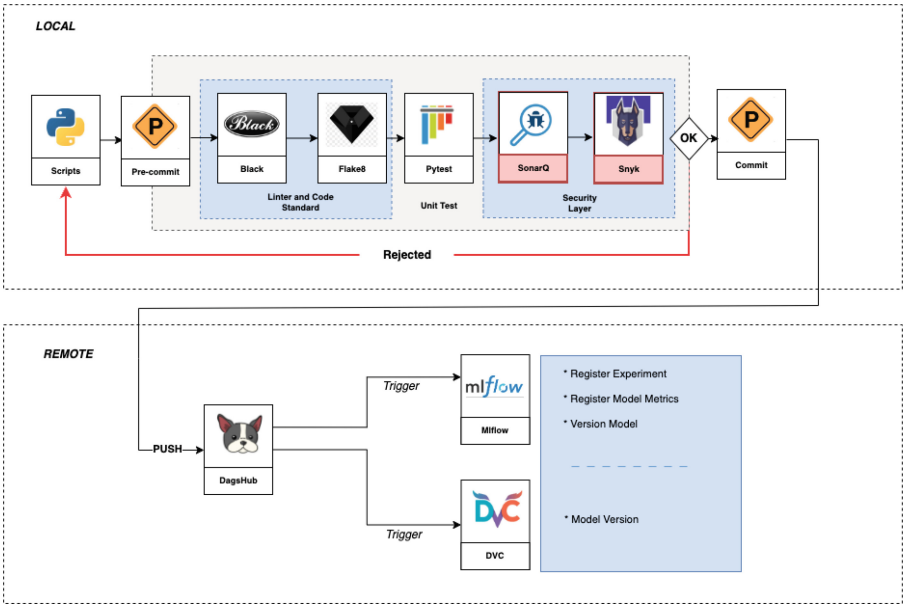

Fig. 2. Specific pipeline in local and remote environment.

tions. DVC complemented this process by providing tight version control over the data sets, which is vital for the reproducibility of the experiments. Once the final model was built and evaluated through rigorous quality and security testing within GitLab, it was hosted on Amazon S3, facilitating safe and effective deployment. This comprehensive pipeline illustrates a robust and systematic methodology, reflecting best practices in MLOps and highlighting the synergy between development tools and industry standards.

The monitoring phase in MLOps is essential to ensure the continued effectiveness of the machine learning model once implemented. This stage provides constant monitoring of the performance and functionality of the model in production, using specific metrics to evaluate its accuracy, the quality of predictions, and the response time of the service. This 24/7 monitoring captures valuable operational insights that are crucial to maintaining the integrity of the model and ensuring its alignment with business objectives. Data collected during monitoring informs the feedback process, identifying opportunities for improvement and optimization of the model and underlying data. This feedback is an integral component of the MLOps lifecycle, facilitating rapid iteration and adaptation to changing data dynamics and environment requirements. In this way, monitoring not only acts as a preventative measure against performance deterioration but also drives the continuous development and evolution of the machine learning model.

Efficient versioning of data and software artifacts is essential in the MLOps ecosystem to ensure the integrity, reproducibility, and traceability of ML models

throughout their lifecycle [15]. In the architecture implemented for this work, GitLab CI/CD for continuous integration and deployment is used. It provides an automated workflow for code and model delivery. In parallel, MLflow is used for versioning and management of ML models, allowing detailed control over different versions of the model that are in different stages, such as production and development stages. This ensures that any version of the model can be deployed or retracted with ease and efficiency.

Additionally, data versioning is handled by DVC, a system that extends Git's versioning capabilities to large data sets. DVC makes it easy to manage versions of the datasets used to train and validate ML models, ensuring that experiments are consistent and that the data used in models in production is correct. Together, these tools form a robust framework for versioning in MLOps, not only providing control over the code and models but also over the data that feeds them, which is crucial for maintaining quality and consistency in the version. Implementation of solutions based on machine learning [15,22].

3.4 Implemented Phases

The documented project reflected an MLOps approach, characterized by an iterative and step-wise process that unfolded in three distinct but interrelated phases. The retrospective details how each phase built on its predecessor, establishing a dynamic synergy between design, development, and operations. This approach allowed the effective implementation of machine learning models, agile adaptation, and continuous improvement throughout the project.

Design Phase. The project's design phase was based on solid principles of safety and operability. It was started with a focus on security, ensuring access control and data integrity, which was essential in establishing the foundation for a secure and reliable ML architecture [19]. Security considerations extended to continuous monitoring, preparing the system for real-time incident detection and response [23].

From an operational perspective, the environment was assessed to ensure model playback capabilities aligned with organizational standards, and meticulously planned for production rollout, with constant monitoring [8]. The design included preparation for ongoing maintenance, ensuring system support and adaptability to respond to future needs and changing requirements. Critical performance factors such as capacity, scalability, fault tolerance, and system usability were also considered, all essential for an effective [8,11] deployment. Finally, compliance with standards and legal requirements was taken into account to align the system design with applicable regulations and regulatory frameworks [23].

Model Development Phase. This phase of the MLOps approach was structured around a meticulously designed data pipeline approach, as illustrated in Figs. 3 and 2. This phase began with the extraction and preparation of raw data,

managed through DVC (Data Version Control), which ensured traceability and efficient management of large volumes of data. The script `make_dataset.py` within a Git repository managed the creation of the dataset, evidencing the integration between data versioning tools and source code (Fig. 4).

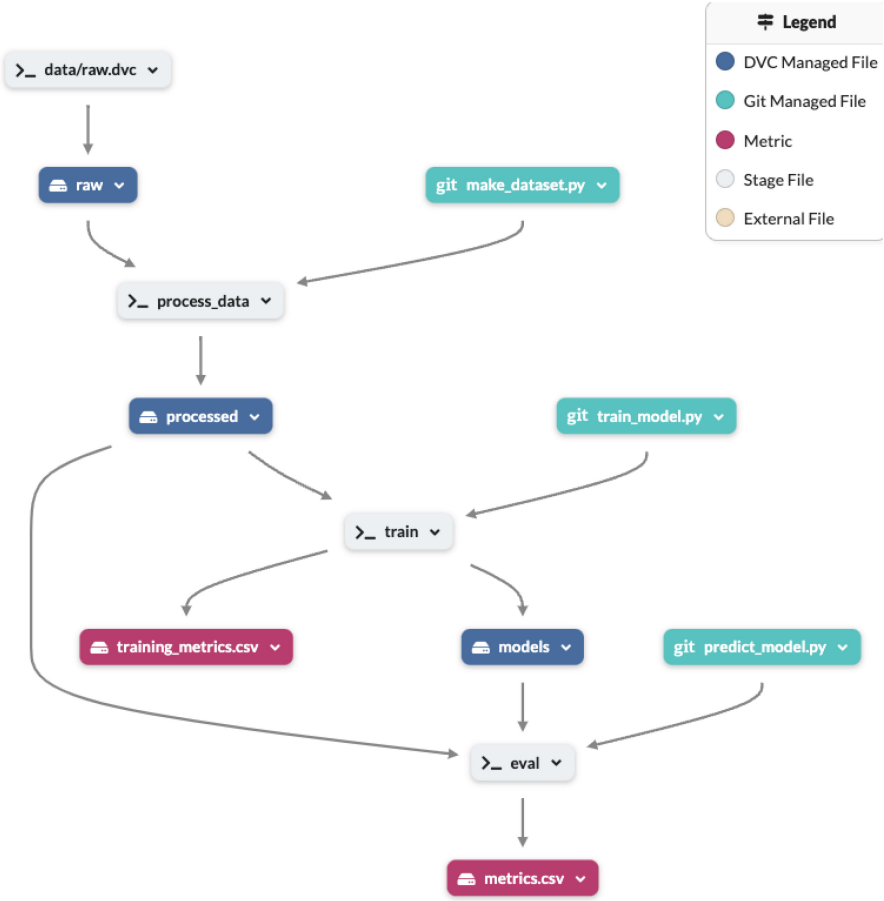

Fig. 3. Execution of model flow.

Subsequently, data processing was run through a script called `process_data.py`, which transformed the raw data into a processed format ready for model training. This processing was crucial for cleaning and preparing the data, essential steps before any predictive analysis. The script `train_model.py` then took over to train the model, with training information stored in the file `training_metrics.csv` capturing the relevant metrics of this process, thus allowing for continuous evaluation and tuning of the model (see Fig. 5).

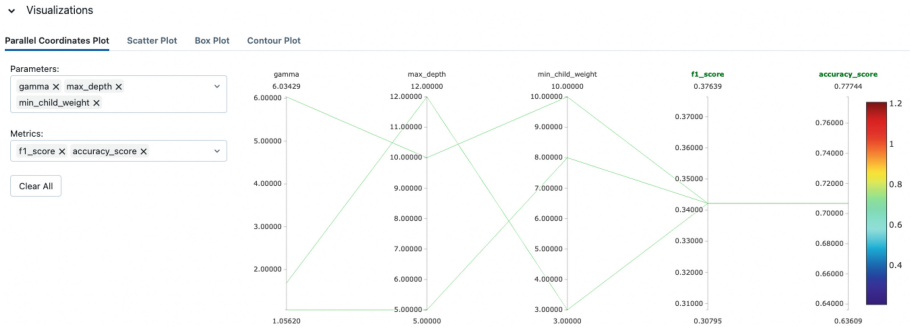

Fig. 4. Control metrics for models.

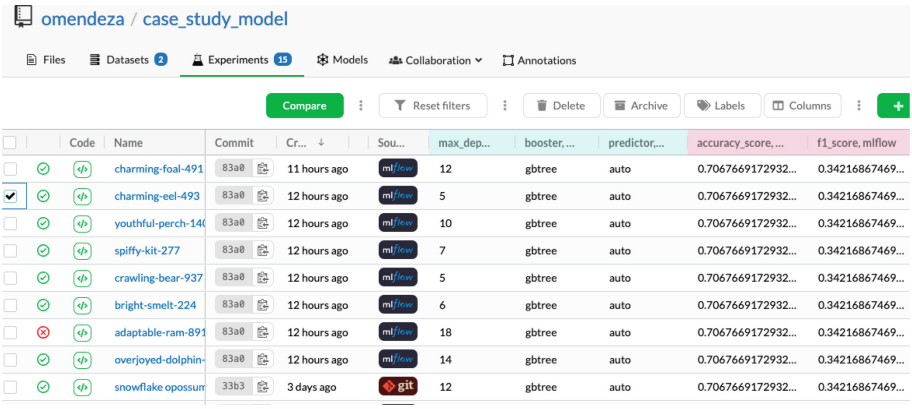

Fig. 5. Model versioning in DAGS.

To close the loop, `predict_model.py` was used to generate predictions and evaluate the model's effectiveness on unseen data, with `metrics.csv` recording the final performance. Visualization and tracking tools, facilitated the comparison of different model runs and configurations, allowing informed selection of the best model based on parameters such as accuracy, and F1 score, among others. The graphical interface provided a clear visual representation of the impact of different hyperparameters on model performance, essential for data-driven decision-making in modern data science (see Figs. 3 and 5).

Operations Phase. The operations phase capitalized on the CI/CD workflow provided by GitLab, which streamlined both testing and production deployments of ML models. The GitLab CI/CD environment made it possible to create code-defined pipelines, handling tasks from data acquisition to model validation, as seen in the works `get_data`, `model_train` and `model_validate` as presented in Fig. 6.

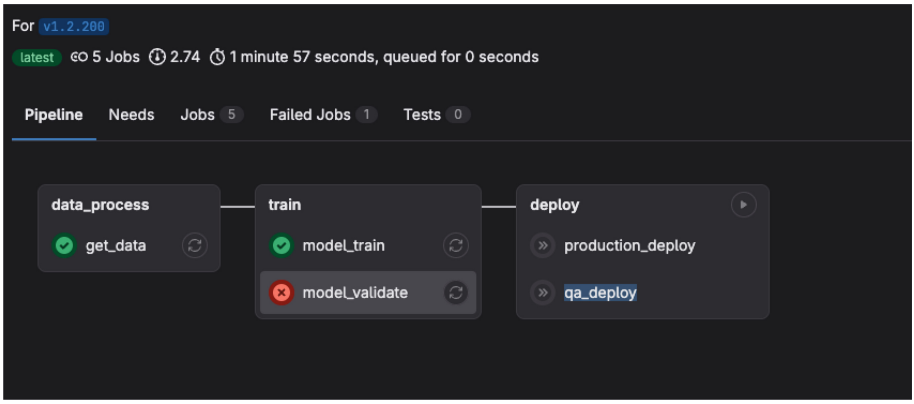

Fig. 6. Gitlab pipeline.

GitLab's tagging functionality allowed detailed management of deployment versions, promoting exhaustive tracking of each iteration and enabling controlled and systematic deployments. This labeling was crucial for maintaining consistency and the ability to react quickly to changes, highlighting the platform's flexibility to adapt and adjust models as necessary. Additionally, version logging in GitLab CI/CD provided a solid foundation for constant monitoring and continuous improvement, ensuring model reproducibility by deploying specific versions to different environments, such as production and QA [3,17].

Regarding security and code quality, automated tools such as Snyk and SonarQube were integrated into the workflow. Snyk focused on proactive identification and resolution of vulnerabilities in dependencies, while SonarQube provided static code analysis to detect bugs and code smells, thus ensuring adherence to programming best practices (see Figs. 7 and 8) [4,18]. These safety and quality tools were implemented at the local pipeline level, operating continuously to maintain safety and quality in each phase of the development cycle.

The influence of these tools on monitoring and continuous improvement was significant. They allowed for constant evaluation of the code and models, highlighting the importance of feedback in the MLOps process. Regarding the reproducibility of the models, the combination of these tools with controlled records and versions allowed the exact reconstruction of any state of the model, facilitating debugging and future development (see Figs. 3, 5, 7, and 8). These practices ensured that the MLOps infrastructure not only stayed aligned with performance and security goals but also continually adapted and improved in response to discoveries and changing production environments.

Phases Integration. The CI/CD pipelines in the MLOps project orchestration stood out for their ability to unify and automate the different stages of the model development life cycle. From data management to deployment and monitoring, each step was articulated through a pipeline designed to ensure the fluidity and accuracy of the process. This pipeline infrastructure not only improved

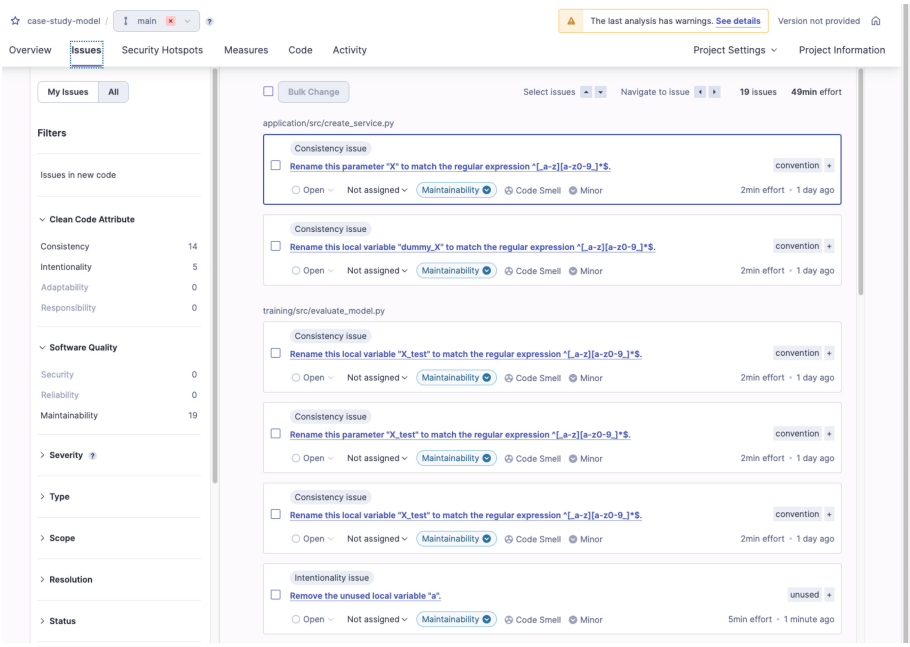

Fig. 7. Sonar monitoring.

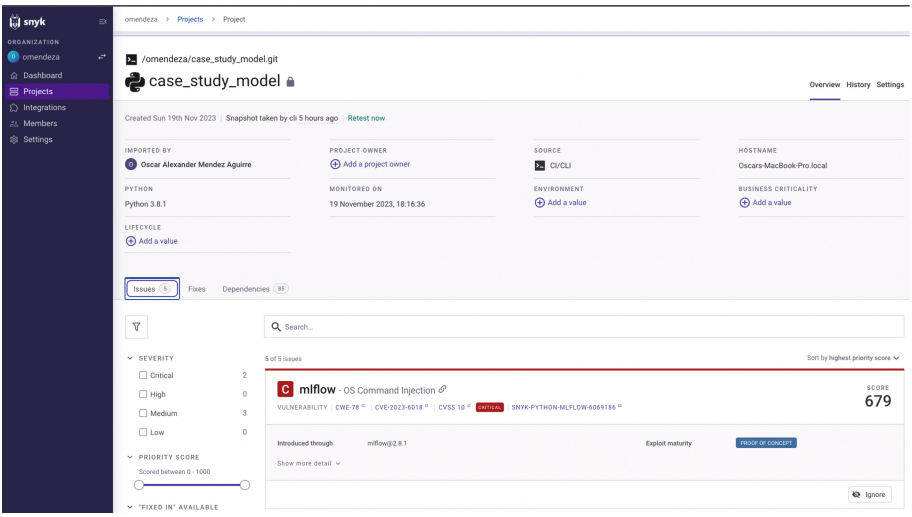

Fig. 8. Snik security monitoring.

operational efficiency but also strengthened the reproducibility of the models by allowing each phase to be exactly replicated or reversed if necessary, thanks to the complete traceability of each deployed version.

The impact of these pipelines extended to system security and scalability. Continuous integration and continuous deployment provided a controlled environment for early detection of vulnerabilities and bugs, improving the security of code and ML models. Additionally, the ability to scale and adapt the models quickly to changes in data or operational requirements underscored the agility of the system. This allowed the project to not only respond to current needs but also be prepared for future demands, evidencing the adaptability and sustainability of the MLOps approach adopted.

4 Conclusions and Future Work

It is essential to highlight that MLOps processes can present greater challenges in dynamic contexts, influenced by factors such as the size of companies and the composition of development, ML, and data analysis teams. This situation underscores the importance of continued exploration of innovative tools and novel concepts that address the automation and application of MLOps in varied domains, especially those with large and complex volumes of data.

In parallel, ongoing research into the application of MLOps and its impact on the security of AI-based systems highlights a critical opportunity for the advancement of knowledge. This area of study promises to accelerate the definition of best practices in safeguarding and improving the integrity of AI models. Considering that AI is an emerging and evolving field, multiple possibilities are identified to research and develop tools that facilitate its effective application.

Additionally, it is suggested to investigate the long-term impact of the implementation of MLOps strategies in different types of industries and companies. Such a study would provide a better understanding of how these frameworks affect operational efficiency and effectiveness, contributing to increased quality, security, scalability, and interoperability in AI-based processes and products.

In future research, it would be fruitful to examine the interaction between MLOps and agile software development methodologies. This synergy has the potential to reveal insights into how agile practices can further optimize MLOps workflows, especially concerning the speed of adaptation and flexibility in changing environments. A detailed study on integrating MLOps with agile approaches could offer valuable guidance for organizations looking to streamline their ML development processes while ensuring quality and efficiency.

Furthermore, another area that deserves attention is the evaluation of the impact of MLOps in democratizing access to AI in less technologically advanced sectors. Investigating how MLOps can be adapted and applied in small and medium-sized enterprises (SMEs) or resource-constrained sectors would offer valuable insight into the universalization of AI technologies. Studies along these lines could identify barriers, propose solutions to overcome them and develop strategies so that these organizations can efficiently take advantage of the advantages of AI, thus contributing to a more equitable distribution of its benefits.

References

1. Bodor, A., Hnida, M., Najima, D.: MLOps: overview of current state and future directions, vol. 629. Springer (2023). https://doi.org/10.1007/978-3-031-26852-6_14
2. Brazhenenko, M., Kozachok, P., Petrivskyi, V., Bychkov, O., Shevchenko, V.: Cloud based architecture design of system of systems. In: International Conference on CAD Systems in Microelectronics (CADSM) (2019). https://doi.org/10.1109/CADSM.2019.8779307
3. Chatterjee, A., Ahmed, B., Hallin, E., Engman, A.: Quality assurance in mlops setting: An industrial perspective. arXiv 3362 (2022)
4. Díaz, J., Pérez, J.E., Lopez-Peña, M.A., Mena, G.A., Yagüe, A.: Self-service cybersecurity monitoring as enabler for devsecops. IEEE Access **7**, 100283–100295 (2019)
5. Eck, B., Kabakci-Zorlu, D., Chen, Y., Savard, F., Bao, X.: A monitoring framework for deployed machine learning models with supply chain examples. arXiv preprint arXiv:2211.06239 (2022)
6. Fujii, T.Y., et al.: A digital twin architecture model applied with mlops techniques to improve short-term energy consumption prediction. Machines **10**(1), 23 (2021)
7. Ghanta, S., et al.: Ml health monitor: Taking the pulse of machine learning algorithms in production. In: Applications of Machine Learning, vol. 11139 (2019). https://doi.org/10.1117/12.2529598
8. Google Cloud: Mlops: Continuous delivery and automation pipelines in machine learning (2022). https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning?hl=es-419
9. Hernandez, J., Daza, K., Florez, H.: Spiking neural network approach based on caenorhabditis elegans worm for classification. IAENG Int. J. Comput. Sci. **49**(4) (2022)
10. Hernandez, J., Florez, H.: An experimental comparison of algorithms for nodes clustering in a neural network of caenorhabditis elegans. In: Gervasi, O., et al. (eds.) ICCSA 2021. LNCS, vol. 12957, pp. 327–339. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-87013-3_25
11. Hewage, N., Meedeniya, D.: Machine learning operations: A survey on mlops tool support. arXiv (2022). https://arxiv.org/abs/2202.10169
12. Jain, A., Malviya, A., Bajaj, D., Bhavsar, R., Savyanavar, A.: Brain tumor detection using mlops and hybrid multi-cloud. In: 2022 IEEE International Conference on Blockchain and Distributed Systems Security, ICBDS 2022 (2022). https://doi.org/10.1109/ICBDS53701.2022.9936020
13. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine learning operations (mlops): Overview, definition, and architecture. IEEE Access (2023)
14. Lima, A., Monteiro, L., Furtado, A.: Mlops: Practices, maturity models, roles, tools, and challenges - a systematic literature review. In: International Conference on Enterprise Information Systems, ICEIS - Proceedings, vol. 1, pp. 308–320 (2022). https://doi.org/10.5220/0010997300003179
15. Makinen, S., Skogstrom, H., Laaksonen, E., Mikkonen, T.: Who needs mlops: what data scientists seek to accomplish and how can mlops help? In: Proceedings - 2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI, WAIN 2021, pp. 109–112 (2021). https://doi.org/10.1109/WAIN52551.2021.00024
16. Malatras, A.: Securing machine learning algorithms (2021). https://www.enisa.europa.eu/publications/securing-machine-learning-algorithms

17. Malhotra, Y.: How you can implement well-architected? zero trust? hybrid-cloud computing beyond? lift and shift?: cloud-enabled digital innovation at scale with infrastructure as code (iac), devsecops and mlops. In: 2022 New York State Cyber Security Conference: Invited Presentations, Albany, New York: https://its.ny.gov/2022-nyscsc (2022)
18. Mathur, O.: Ai/security-and-privacy-guide (2023). https://owasp.org/www-project-ai-security-and-privacy-guide/owaspaiexchange.html
19. MLOps Community: Summary of mlops principles and best practices (2022). https://ml-ops.org/content/mlops-principles#summary-of-mlops-principles-and-best-practices
20. Moreschini, S.: Applications of mlops in the cognitive cloud continuum. In: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics). LNCS, vol. 13709, pp. 650–655 (2022). https://doi.org/10.1007/978-3-031-21388-5_51
21. Paul, S.K., Riaz, S., Das, S.: A conceptual architecture for ai in supply chain risk management. In: TENCON 2022-2022 IEEE Region 10 Conference (TENCON), pp. 1–5. IEEE (2022)
22. Ranawana, R., Karunananda, A.: An agile software development life cycle model for machine learning application development. In: 5th SLAAI - International Conference on Artificial Intelligence and 17th Annual Sessions, SLAAI-ICAI 2021 (2021). https://doi.org/10.1109/SLAAI-ICAI54477.2021.9664736
23. Robertson, J., Robertson, S.: Volere. Requirements Specification Templates (2000)
24. Sculley, D., et al.: Hidden technical debt in machine learning systems. Advances in neural information processing systems **28** (2015)
25. Symeonidis, G., Nerantzis, E., Kazakis, A., Papakostas, G.A.: Mlops – definitions, tools and challenges. arXiv (2022). https://arxiv.org/abs/2201.00162
26. Symeonidis, G., Nerantzis, E., Kazakis, A., Papakostas, G.: Mlops - definitions, tools and challenges. In: 2022 IEEE 12th Annual Computing and Communication Workshop and Conference, CCWC 2022, pp. 453–460 (2022). https://doi.org/10.1109/CCWC54503.2022.9720902
27. Velosa, F., Florez, H.: Edge solution with machine learning and open data to interpret signs for people with visual disability. **2714**, 15–26 (2020)
28. Zhang, X., Jaskolka, J.: Conceptualizing the secure machine learning operations (secmlops) paradigm. In: 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), pp. 127–138. IEEE (2022)